DBMS Model Question Paper- I

Module – 1

Q.1 a Explain the types of end users with examples.

---

1. Casual End Users

- **Description:** These users occasionally access the database but do not have detailed knowledge of its structure. They typically use query languages to retrieve information.
- **Example:** A manager who occasionally queries the database to generate a sales report.

2. Naive or Parametric End Users

- **Description:** These users frequently use the database but have a limited understanding of its complexities. They often interact with the system through pre-defined functions or transactions.
- **Example:** A bank teller who uses a banking application to enter customer transactions and retrieve account information.

3. Sophisticated End Users

- **Description:** These users have a good understanding of the database and its capabilities. They interact with the database to perform complex queries and analyses.
- **Example:** A data analyst who writes SQL queries to extract, manipulate, and analyze large datasets for business insights.

4. Stand-alone Users

- **Description:** These users maintain personal databases using tools like Microsoft Access or SQLite. They create and manage their databases independently.
- **Example:** A small business owner who uses Microsoft Access to manage customer information and sales records.

5. System Analysts and Application Programmers

- **Description:** These users design, implement, and maintain database applications. They write application programs that interact with the database and ensure it meets the needs of the organization.
- **Example:** A software developer who creates and maintains an inventory management system that interfaces with a company's database.

6. Database Administrators (DBAs)

VTU360.IN

- **Description:** These users are responsible for the overall management of the database. They ensure its integrity, security, and performance, and manage user access.
- **Example:** A DBA who manages database backup and recovery, configures security settings, and optimizes database performance.

Q1 b.What are the advantages of using DBMS? Explain.

Using a Database Management System (DBMS) has numerous advantages. Here are some of the key benefits:

1. Data Redundancy and Inconsistency Control:

- **Redundancy Minimization**: A DBMS reduces data redundancy by integrating all the data into a single database, reducing the need for duplicate data.
- **Inconsistency Reduction**: By centralizing data storage, a DBMS ensures that updates are consistent across the system, preventing data anomalies.

2. Data Sharing:

- **Multi-User Environment**: A DBMS allows multiple users to access and work with the data simultaneously.
- **Controlled Access**: Users can be granted different levels of access, ensuring data security and integrity.

3. Improved Data Security:

- **Access Controls**: DBMSs provide robust mechanisms for controlling user access and permissions.
- **Encryption**: Sensitive data can be encrypted to protect it from unauthorized access.

4. Data Integrity:

- **Constraints**: Integrity constraints such as primary keys, foreign keys, and unique constraints help maintain the accuracy and consistency of data.
- **Consistency Rules**: DBMSs enforce data consistency rules automatically, reducing the chances of errors.

5. Backup and Recovery:

- **Automatic Backup**: DBMSs often include tools for automatic data backup, ensuring that data can be recovered in case of a failure.
- **Recovery Mechanisms**: DBMSs provide recovery solutions to restore data after crashes, ensuring minimal data loss.

VTU360.IN

6. Data Independence:

- **Logical Data Independence**: Changes to the logical schema do not affect the application programs.
- **Physical Data Independence**: Changes to the physical storage of data do not affect the logical schema or application programs.

7. Efficient Data Access:

- **Query Optimization**: DBMSs use sophisticated algorithms to optimize query performance, making data retrieval efficient.
- **Indexing**: Indexes can be created to improve the speed of data retrieval operations.

8. Concurrent Access:

- **Concurrency Control**: DBMSs handle concurrent access to data, ensuring that multiple users can perform transactions simultaneously without conflicts.
- **Locking Mechanisms**: These prevent conflicts and ensure data consistency during concurrent operations.

9. Enhanced Data Integration:

- **Centralized Data Management**: Data from various sources can be integrated into a single database, providing a unified view of the data.
- **Data Warehousing**: DBMSs can support data warehousing for better data analysis and reporting.

10. Application Development Efficiency:

- **Standardized Query Language**: SQL provides a standardized language for querying and managing data, making it easier to develop and maintain applications.
- **DBMS Tools**: Many DBMSs offer tools and interfaces that simplify the development process.

11. Cost Efficiency:

- **Reduced Storage Costs**: By minimizing redundancy, storage costs are reduced.
- **Maintenance Savings**: Centralized data management reduces the cost and effort involved in maintaining data.

12. Improved Decision Making:

- **Data Analysis**: A DBMS can provide advanced data analysis tools, enabling better decision-making through data insights.

VTUSO.IN

- **Reporting**: Various reporting tools integrated into DBMSs help generate insightful reports for decision-makers.

Q1.c Describe the characteristics of database.

Databases are fundamental components in modern computing and information management. Here are some key characteristics of databases:

1. **Structured Data Storage**: Databases organize data into tables or collections with predefined schemas. This structure helps in efficiently storing, retrieving, and managing data.
2. **Data Integrity**: Databases enforce rules and constraints to ensure the accuracy and consistency of the data. Common integrity constraints include primary keys (which uniquely identify records), foreign keys (which establish relationships between tables), and various data validation rules.
3. **Data Management**: Databases support various data manipulation operations such as creating, reading, updating, and deleting data (often abbreviated as CRUD operations). This functionality is facilitated through database management systems (DBMS) and query languages like SQL.
4. **Concurrency Control**: Databases manage concurrent access by multiple users or processes to ensure that transactions are processed reliably without conflicts. This involves mechanisms for locking data and managing transactions.
5. **Transaction Management**: Databases support transactions, which are sequences of operations that are executed as a single unit. Transactions are designed to ensure that operations are completed fully and correctly or not executed at all, maintaining data consistency (following the ACID properties: Atomicity, Consistency, Isolation, Durability).
6. **Scalability and Performance**: Databases are designed to handle large volumes of data and high transaction rates efficiently. They use indexing, optimization techniques, and caching to improve performance and scalability.
7. **Data Security**: Databases implement security measures to protect data from unauthorized access and breaches. This includes user authentication, authorization, encryption, and auditing.
8. **Backup and Recovery**: Databases provide mechanisms for backing up data and recovering it in case of failures or data loss. Regular backups and recovery plans are essential for data protection and continuity.
9. **Data Retrieval and Querying**: Databases offer powerful querying capabilities to retrieve and manipulate data. Query languages like SQL enable users to perform complex searches, aggregations, and data analysis.
10. **Data Relationships**: Databases can represent complex relationships between data entities through mechanisms such as foreign keys and joins. This allows for the efficient organization and retrieval of related information.

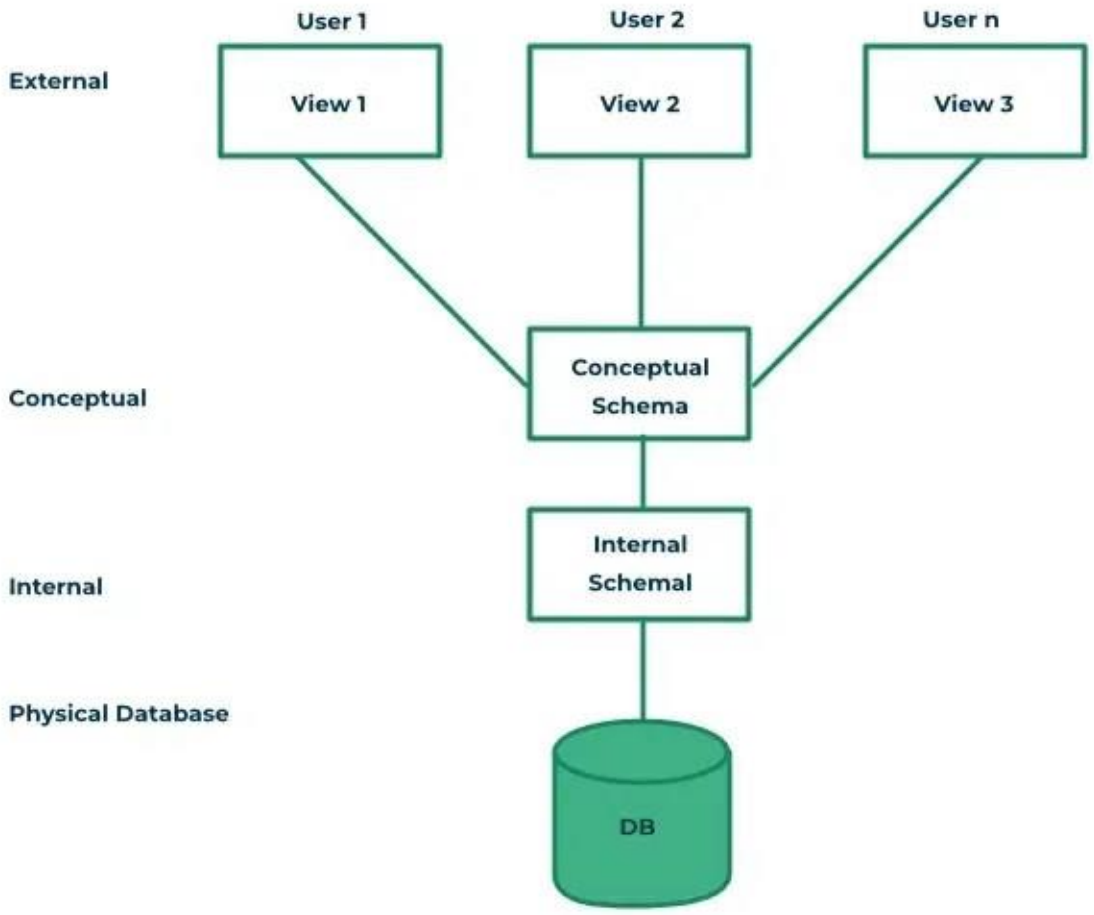
ignou.in

11. **Flexibility and Extensibility**: Many databases are designed to be flexible and can adapt to changes in data requirements. Some modern databases support schema evolution and dynamic changes.
12. **Data Redundancy Reduction**: Through normalization processes, databases reduce redundancy and improve data consistency by organizing data into related tables and minimizing duplication.

Q2.a Explain three schema architecture. Why mappings b/w schema levels are required?

The three-schema architecture is a framework used in database systems to manage data abstraction and separation. It involves three distinct levels of schemas:

VTU360.IN



1. **Internal Schema (Physical Schema)**:
   - **Definition**: This schema describes how data is physically stored in the database. It deals with the storage structure, file organization, indexing, and access methods.
   - **Purpose**: It provides the details about how data is organized and managed on the storage medium, ensuring efficient data retrieval and management.
   - **Focus**: Implementation details, such as storage devices, data structures, and access paths.
2. **Conceptual Schema (Logical Schema)**:
   - **Definition**: This schema represents the entire database's logical structure. It defines the database's entities, relationships, and constraints without considering how they are physically stored. ○ **Purpose**: It provides a unified and coherent view of the data, ensuring consistency and integrity across the database. It serves as an abstraction layer between the physical storage and user views. ○ **Focus**:

Data organization, relationships, and constraints without concern for physical storage details.

3. **External Schema (View Schema)**:
    - **Definition**: This schema defines different user views or perspectives of the database. Each view is a subset of the database that is tailored to specific user needs or application requirements.
    - **Purpose**: It provides customized views of the data for different users or applications, enhancing security and usability by restricting access to only the relevant parts of the database.
    - **Focus**: User interactions and data presentation tailored to specific needs.

Importance of Mappings Between Schema Levels

Mappings between these schema levels are essential for several reasons:

1. **Data Abstraction**:
    - **Purpose**: Mappings allow for data abstraction, separating the physical storage of data from its logical representation and user views. This separation ensures that changes to the physical storage (e.g., changing indexing methods or storage formats) do not impact the logical schema or user views.
2. **Flexibility and Independence**:
    - **Purpose**: By defining mappings between schemas, databases can evolve independently at different levels. For instance, if the internal schema changes (e.g., due to hardware upgrades), the conceptual schema and external schemas can remain unchanged, preserving the consistency of user views and logical data organization.
3. **Consistency and Integrity**:
    - **Purpose**: Mappings ensure that the logical schema accurately reflects the data stored at the physical level and that user views are consistently represented. This helps in maintaining data integrity across various schemas.
4. **User-Specific Views**:
    - **Purpose**: Mappings allow different users to interact with the database according to their needs without affecting the overall database structure. Users can work with customized views (external schemas) tailored to their specific roles or applications, while the underlying data structure (conceptual schema) remains consistent.
5. **Simplified Management**:

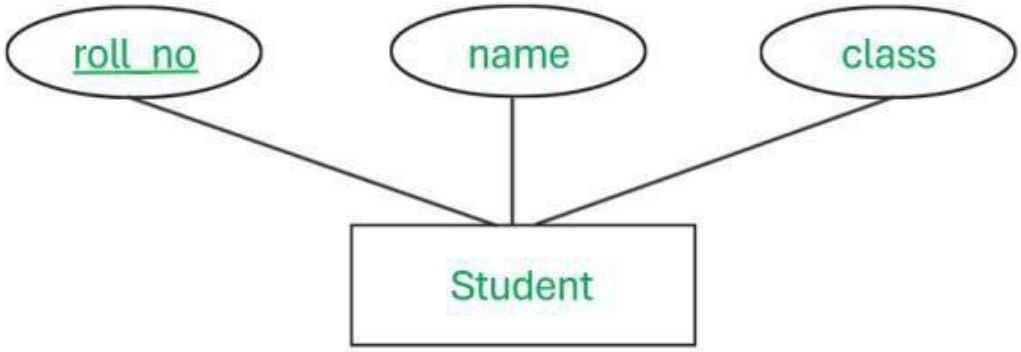
    - **Purpose**: Managing and maintaining the database becomes easier when different schemas are mapped correctly. Database administrators can optimize physical storage and indexing without altering the logical schema or user views.

Q2.b Explain the different types of attributes in ER model.

VTUBOOK.IN

1. Simple Attribute

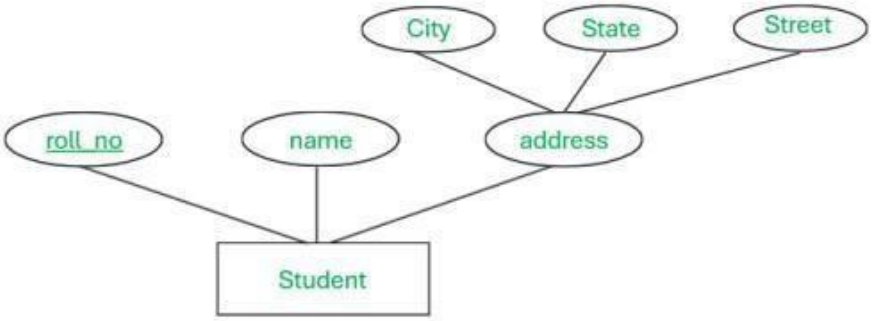
An attribute that cannot be further subdivided into components is a simple attribute.
Example: The roll number of a student, the ID number of an employee, gender, and many more.



Simple Attribute

2.Composite Attribute

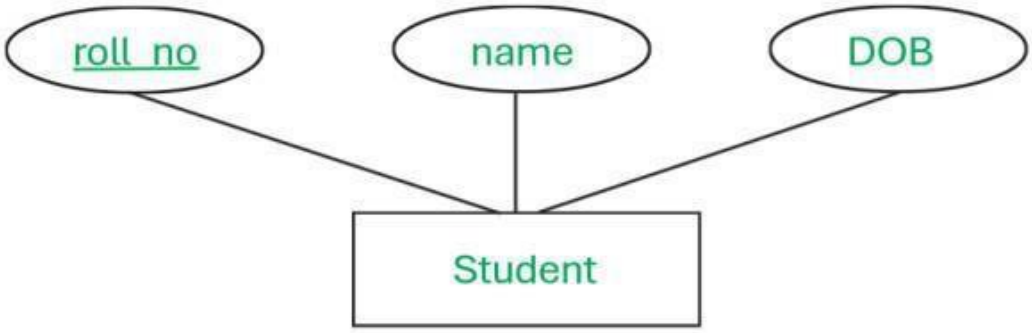
An attribute that can be split into components is a composite attribute. **Example:** The address can be further split into house number, street number, city, state, country, and pin code, the name can also be split into first name middle name, and last name.



Composite Attribute

3. Single-Valued Attribute

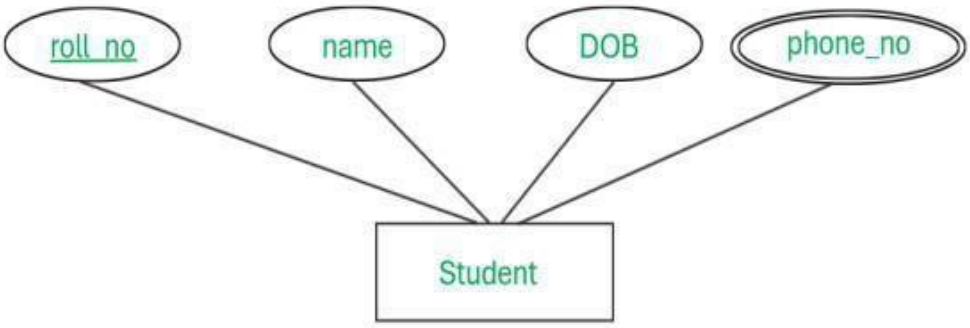
The attribute which takes up only a single value for each entity instance is a single-valued attribute.
Example: The age of a student, Aadhar card number.



Single-Valued

4. Multi-Valued Attribute

The attribute which takes up more than a single value for each entity instance is a multi-valued attribute. And it is represented by double oval shape. **Example:** Phone number of a student: Landline and mobile.

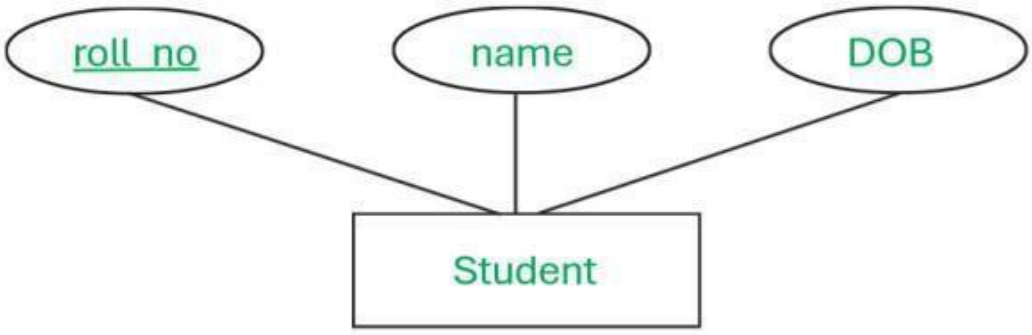


Multi-valued

5. Stored Attribute

The stored attribute are those attribute which doesn't require any type of further update since they are

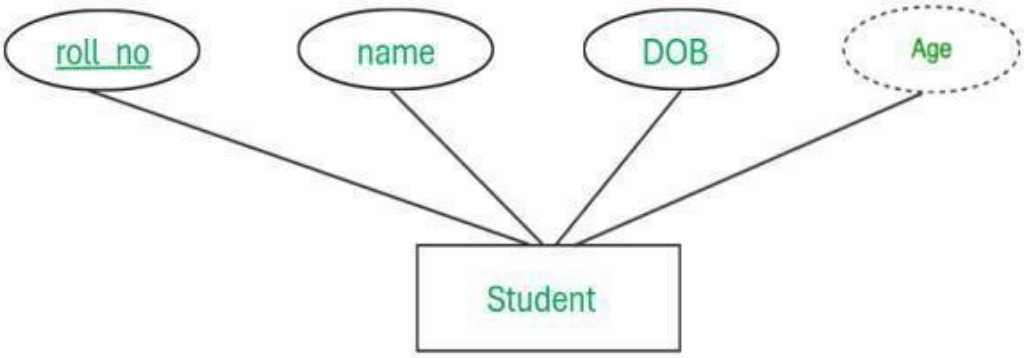
stored in the database. **Example:** DOB(Date of birth) is the stored attribute.



Stored-attribute

6. Derived Attribute

An attribute that can be derived from other attributes is derived attributes. And it is represented by dotted oval shape.
Example: Total and average marks of a student, age of an employee that is derived from date of birth.



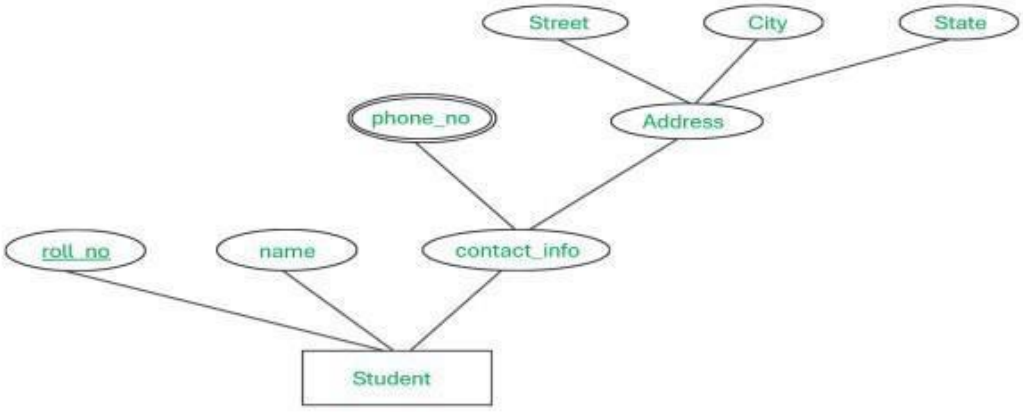
Derived-attribute

7. Complex Attribute

Those attributes, which can be formed by the nesting of composite and multivalued attributes, are called "*Complex Attributes*". These attributes are rarely used in DBMS(DataBase

Management System). That's why they are not so popular.

Example: Address because address contain composite value like street, city, state, PIN code and also multivalued because one people has more that one house address.

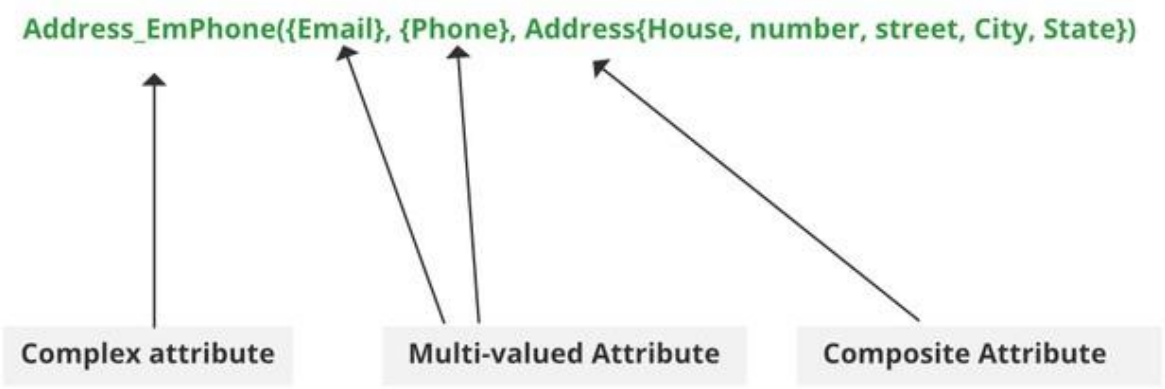


Complex-attribute

Representation

Complex attributes are the nesting of two or more composite and multi-valued attributes. Therefore, these multi-valued and composite attributes are called 'Components' of complex attributes.

These components are grouped between parentheses '( )' and multi-valued attributes between curly braces '{ }', Components are separated by commas ', '. For example: let us consider a person having multiple phone numbers, emails, and an address. Here, phone number and email are examples of multi-valued attributes and address is an example of the composite attribute, because it can be divided into house number, street, city, and state.



Complex attributes

Components

Email, Phone number, Address(All components are separated by commas and multi-valued are represented between curly braces).

Complex Attribute: Address_EmPhone(You can choose any name).

8. Key attribute

Key attributes are those attributes that can uniquely identify the entity in the entity set.

Example: Roll-No is the key attribute because it can uniquely identify the student.

9. Null Attribute

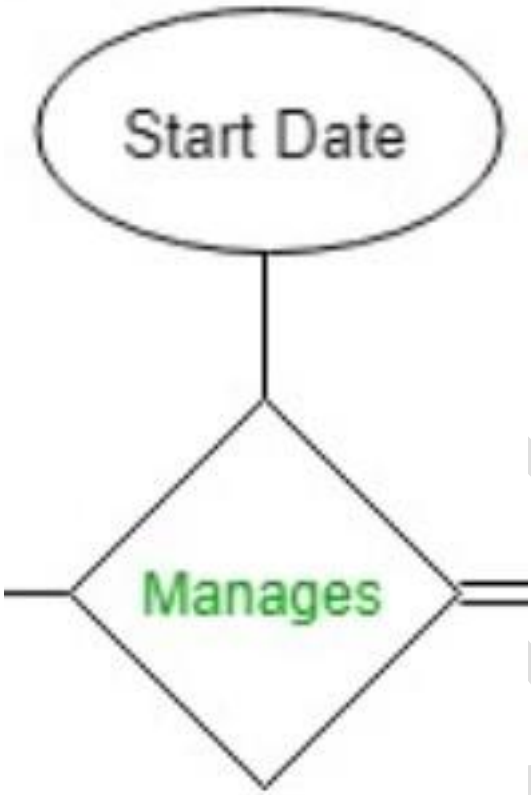
This attribute can take NULL value when entity does not have value for it. **Example** –The 'Net Banking Active Bin' attribute gives weather particular customer having net banking facility activated or not activated.

For bank customer which does not offer facility of net banking in table 'Net Banking Active Bin' attribute is always null till Net banking facility is not activated as this attribute indicates Bank offers net banking facility or does not offers.

10. Descriptive Attribute

Descriptive attribute give information about the relationship set example given below. Here Start Date is the descriptive attribute of Manages relationship.

VTU360.in



Descriptive-Attribute

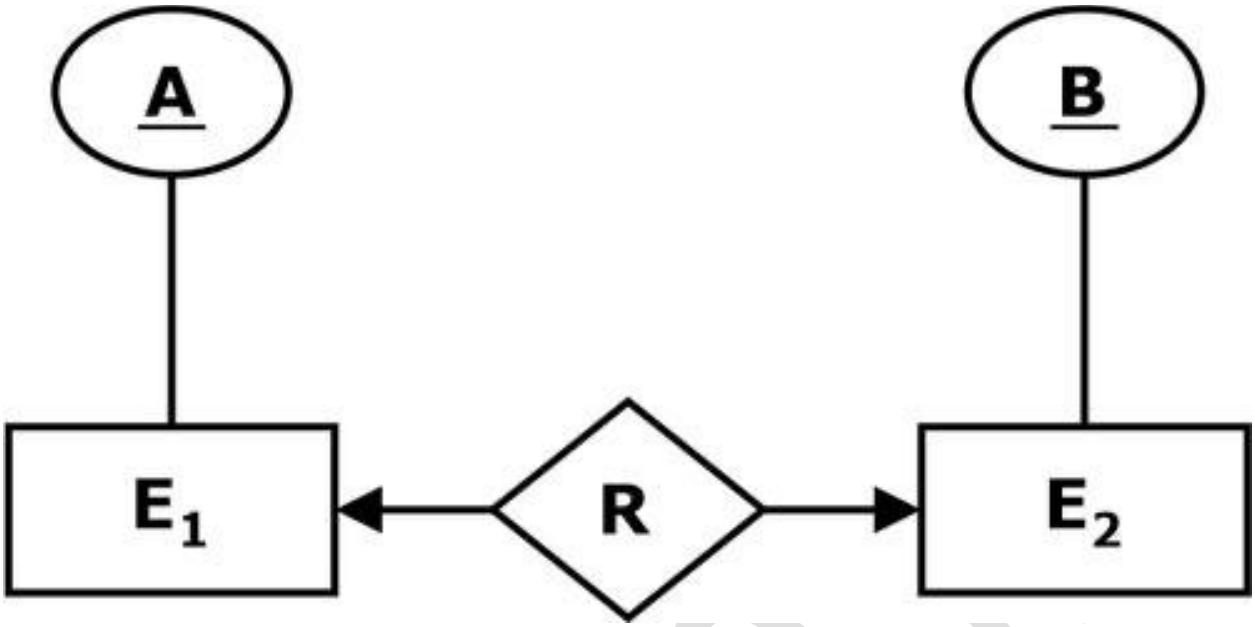
Q2.cExplain the following.

1. Cardinality Ratio 2. Weal Entity

---

1. Cardinality Ratio

VTUSC.IN



Definition: Cardinality ratio describes the number of instances of one entity that are associated with the number of instances of another entity in a relationship. It specifies how many instances of an entity participate in a relationship with instances of another entity.

Types of Cardinality Ratios:

1. **One-to-One (1:1)**:
    - **Definition**: An instance of entity A is associated with at most one instance of entity B, and vice versa.
    - **Example**: In a database of employees and their company-issued cars, each employee might be assigned exactly one car, and each car is assigned to exactly one employee.
2. **One-to-Many (1)**:
    - **Definition**: An instance of entity A can be associated with multiple instances of entity B, but an instance of entity B is associated with at most one instance of entity A.
    - **Example**: A single department (entity A) can have multiple employees (entity B), but each employee belongs to only one department.
3. **Many-to-One (N:1)**:
    - **Definition**: Multiple instances of entity A can be associated with a single instance of entity B, but each instance of entity B is associated with at most one instance of entity A.
    - **Example**: Many employees (entity A) can work under one manager (entity B), but each manager supervises only one department (entity B).

VTU360

4. **Many-to-Many (M**

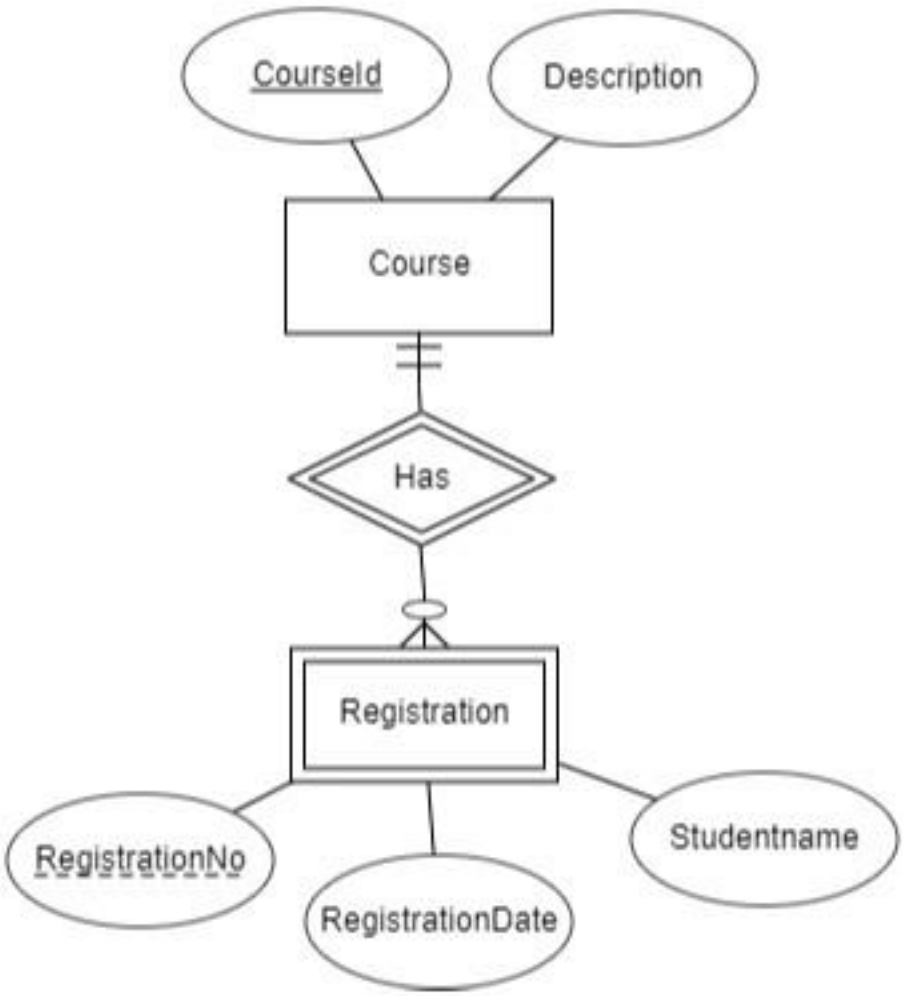
   ):

   - **Definition**: Multiple instances of entity A can be associated with multiple instances of entity B, and vice versa.
   - **Example**: Students (entity A) can enroll in multiple courses (entity B), and each course can have multiple students.

Importance: Cardinality ratios are crucial for understanding how entities relate to each other, which helps in designing the database schema and ensuring that relationships are represented correctly.

VTU360.IN

2. Weak Entity



Definition: A weak entity is an entity that cannot be uniquely identified by its own attributes alone. It relies on a "strong" or "owner" entity and a relationship with that entity to ensure its uniqueness.

Characteristics of Weak Entities:

1. **Lack of Key Attribute**: A weak entity does not have a primary key of its own. Instead, it has a partial key, which is an attribute or set of attributes that can uniquely identify the weak entity in conjunction with the primary key of the strong entity.
2. **Dependence on Strong Entity**: The weak entity is dependent on a strong entity, which provides part of the identifying information. This relationship is typically depicted by a double rectangle (for the weak entity) and a double diamond (for the identifying relationship) in ER diagrams.

3. **Existence Dependency**: A weak entity cannot exist without its associated strong entity. If the strong entity is deleted, the weak entity is also deleted.

Example: Consider an example involving an entity `Order` and a weak entity `OrderItem`. An `Order` entity can have multiple `OrderItem` entities. Here, `OrderItem` may not have a unique identifier on its own; it relies on the `Order` entity's primary key plus a partial key (e.g., `ItemNumber`) to identify each `OrderItem`.

ER Diagram Representation:

- **Weak Entity**: Represented by a double rectangle.
- **Identifying Relationship**: Represented by a double diamond.
- **Partial Key**: Underlined with a dashed line.

Importance: Weak entities are important for representing complex real-world scenarios where certain entities cannot be uniquely identified without referencing another entity. They help in modeling hierarchical or dependent relationships in a database schema.

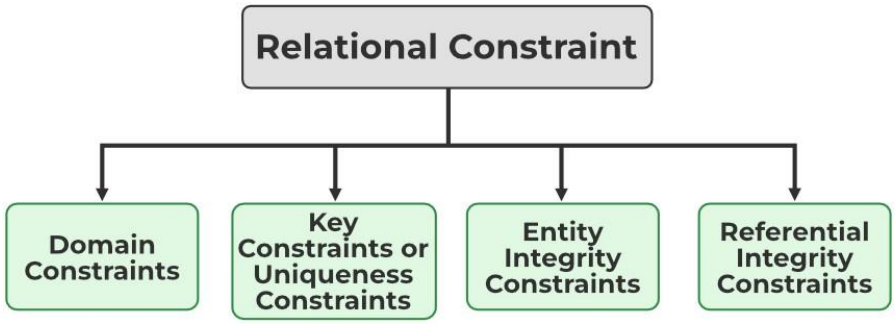
Q3.a Explain the different Relational Model constraint

Rela%onal Constraints

These are the restrictions or sets of rules imposed on the database contents. It validates the quality of the database. It validates the various operations like data insertion, updation, and other processes that have to be performed without affecting the integrity of the data. It protects us against threats/damages to the database. Mainly Constraints on the relational database are of 4 types

- Domain constraints
- Key constraints or Uniqueness Constraints
- En4ty Integrity constraints
- Referen4al integrity constraints

VTUBOOK.IN



1. Domain Constraints

- Every domain must contain atomic values(smallest indivisible units) which means composite and multi-valued attributes are not allowed.
- We perform a datatype check here, which means when we assign a data type to a column we limit the values that it can contain. Eg. If we assign the datatype of attribute age as int, we can't give it values other than int datatype.

Example:

EID	Name	Phone
01	Bikash Dutta	123456789 234456678

Explanation: In the above relation, Name is a composite attribute and Phone is a multi-values attribute, so it is violating domain constraint.

2. Key Constraints or Uniqueness Constraints

- These are called uniqueness constraints since it ensures that every tuple in the relation should be unique.
- A relation can have multiple keys or candidate keys(minimal superkey), out of which we choose one of the keys as the primary key, we don't have any restriction on choosing the primary key out of candidate keys, but it is suggested to go with the candidate key with less number of attributes.
- Null values are not allowed in the primary key, hence Not Null constraint is also part of the key constraint. **Example:**

EID	Name	Phone
01	Bikash	6000000009
02	Paul	9000090009
01	Tuhin	9234567892

Explanation: In the above table, EID is the primary key, and the first and the last tuple have the same value in EID ie 01, so it is violating the key constraint.

3.Entity Integrity Constraints

- Entity Integrity constraints say that no primary key can take a NULL value, since using the primary key we identify each tuple uniquely in a relation.

Example:

EID	Name	Phone
01	Bikash	9000900099
02	Paul	600000009
NULL	Sony	9234567892

Explanation: In the above relation, EID is made the primary key, and the primary key can't take NULL values but in the third tuple, the primary key is null, so it is violating Entity Integrity constraints.

4. Referential Integrity Constraints

- The Referential integrity constraint is specified between two relations or tables and used to maintain the consistency among the tuples in two relations.
- This constraint is enforced through a foreign key, when an attribute in the foreign key of relation R1 has the same domain(s) as the primary key of relation R2, then the foreign key of R1 is said to reference or refer to the primary key of relation R2.
- The values of the foreign key in a tuple of relation R1 can either take the values of the primary key for some tuple in relation R2, or can take NULL values, but can't be empty.

Example:

EID	Name	DNO
01	Divine	12
02	Dino	22
04	Vivian	14
DNO	**Place**	
12	Jaipur	
13	Mumbai	
14	Delhi	

Explanation: In the above tables, the DNO of Table 1 is the foreign key, and DNO in Table 2 is the primary key. DNO = 22 in the foreign key of Table 1 is not allowed because DNO = 22 is not defined in the primary key of table 2. Therefore, Referential integrity constraints are violated here.

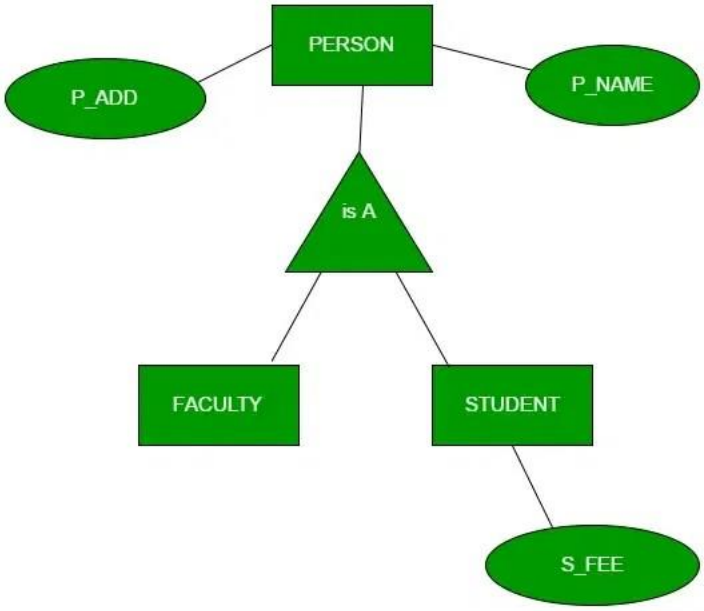
VTUSO.IN

Q3.bDemonstrate the concepts of Generalization & Specialization with examples.

Generalization

Generalization is the process of extracting common properties from a set of entities and creating a generalized entity from it. It is a bottom-up approach in which two or more entities can be generalized to a higher-level entity if they have some attributes in common. For Example, STUDENT and FACULTY can be generalized to a higher-level entity called PERSON as shown in Figure 1. In this case, common attributes like P_NAME, and P_ADD become part of a higher entity (PERSON), and specialized attributes like S_FEE become part of a specialized entity (STUDENT).

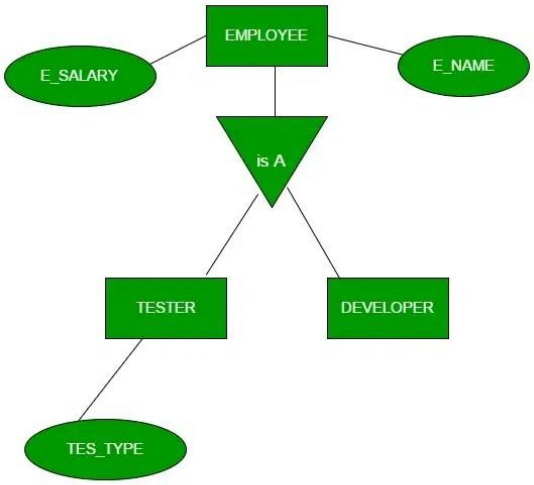
Generalization is also called as ‘ Bottom-up approach”.



Generalization

Specialization

In specialization, an entity is divided into sub-entities based on its characteristics. It is a top-down approach where the higher-level entity is specialized into two or more lower-level entities. For Example, an EMPLOYEE entity in an Employee management system can be specialized into DEVELOPER, TESTER, etc. as shown in Figure 2. In this case, common attributes like E_NAME, E_SAL, etc. become part of a higher entity (EMPLOYEE), and specialized attributes like TES_TYPE become part of a specialized entity (TESTER). Specialization is also called as ” Top-Down approch”.



Specialization

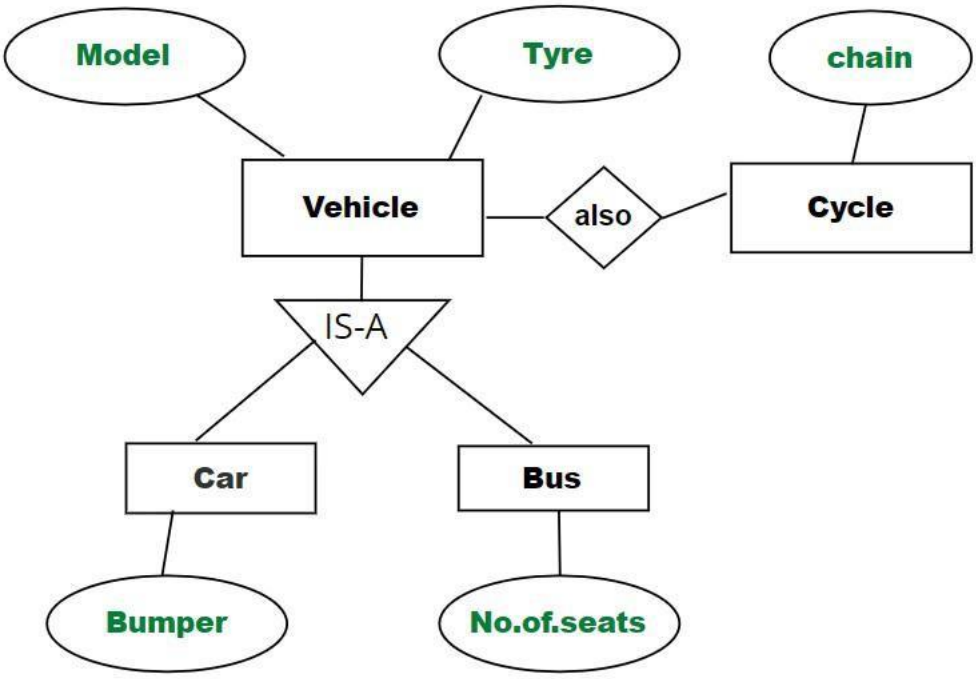
Specialization

Inheritance: It is an important feature of generalization and specialization

- **Attribute inheritance**: allows lower level entities to inherit the attributes of higher level entities and vice versa.
- in diagram: **Car** entity is an inheritance of **Vehicle** entity ,So Car can acquire attributes of **Vehicle** example:car can acquire **Model** attribute of **Vehicle**.

- **Participation inheritance**: In participation inheritance, relationships involving higher level entity set also inherited by lower level entity and vice versa.
- in diagram: Vehicle entity has an relationship with Cycle entity ,So **Cycle entity** can acquire attributes of lower level entities i.e **Car** and **Bus** since it is inheritance of **Vehicle.**

VTU360.IN



Q3.C c Explain Entity Integrity Constraint & Referential Integrity Constraints? Whyeach of these is important in a database.

Entity integrity constraints:

Entity integrity constraints state that primary key can never contain null value because primary key is used to determine individual rows in a relation uniquely, if primary key contains null value then we cannot identify those rows. A table can contain null value in it except primary key field.

Example:

It is not allowed because it is containing primary key as NULL value.

Student_id	Name	Semester	Age
21CSE101	Ramesh	5th	20
21CSE102	Kamlesh	5th	21

21CSE103	Aakash	5th	22
	Mukesh	5th	20

Referential integrity constraints:

It can be specified between two tables. In case of referential integrity constraints, if a Foreign key in Table 1 refers to Primary key of Table 2 then every value of the Foreign key in Table 1 must be null or available in Table 2.

Example:

Here, in below example Block_No 22 entry is not allowed because it is not present in 2nd table.

Student_id	Name	Semester	Block_No
22CSE101	Ramesh	5th	20
21CSE105	Kamlesh	6th	21
22CSE102	Aakash	5th	20
23CSE106	Mukesh	2nd	22
Block_No	**Block Location**		
20	Chandigarh		
21	Punjab		
25	Delhi		

Importance:

- **Data Consistency**: Ensures that relationships between tables are preserved, and that references are valid. This prevents orphaned records and maintains data accuracy.
- **Preventing Invalid Data**: Helps prevent the entry of data that would violate the established relationships between tables, such as entering an order for a non-existent customer.
- **Enforcing Referential Actions**: Defines what happens when a referenced record is updated or deleted. For instance, it can specify that related orders should be deleted (cascade delete) or that the foreign key should be set to null (set null) when a customer record is deleted.

Q4.a Consider the Sailors-Boats-Reserves DB described

s (sid, sname, rating, age) b (bid, bname, color) r (sid,

bid, date)

Write each of the following queries in SQL.

1. Find the colors of boats reserved by Alber.

2. Find all sailor ids of sailors who have a rating of at least 8 or reserved boat 103.

VTU360.IN

3. Find the names of sailors who have not reserved a boat whose name contains the string "storm". Order the names in ascending order.

4. Find the sailor ids of sailors with age over 20 who have not reserved a boat whose name includes the string "thunder".

---

1. Find the colors of boats reserved by Alber.

To find the colors of boats reserved by a sailor named "Alber," you need to join the `Sailors`, `Reserves`, and `Boats` tables. First, identify the `sid` for the sailor named "Alber," and then use it to find the colors of the boats reserved by this sailor.

```
SELECT DISTINCT b.color

FROM Sailors s

JOIN Reserves r ON s.sid = r.sid

JOIN Boats b ON r.bid = b.bid

WHERE s.sname = 'Alber';
```

2. Find all sailor ids of sailors who have a rating of at least 8 or reserved boat 103.

To find all sailor ids of sailors who either have a rating of at least 8 or have reserved boat 103, you need to combine results based on these two conditions.

```
SELECT DISTINCT s.sid
FROM Sailors s

LEFT JOIN Reserves r ON s.sid = r.sid

WHERE s.rating >= 8 OR r.bid = 103;
```

3. Find the names of sailors who have not reserved a boat whose name contains the string "storm". Order the names in ascending order.

To find the names of sailors who have not reserved any boat with a name containing "storm," you need to use a subquery to exclude sailors who have reserved such boats.

```
SELECT DISTINCT s.sname

FROM Sailors s
```

VTU360.IN

```
WHERE s.sid NOT IN (
    SELECT r.sid
    FROM Reserves r
    JOIN Boats b ON r.bid = b.bid
    WHERE b.bname LIKE '%storm%'
)
ORDER BY s.sname ASC;
```

4. Find the sailor ids of sailors with age over 20 who have not reserved a boat whose name includes the string “thunder”.

To find the sailor ids of sailors older than 20 who have not reserved a boat with a name containing "thunder," you need a subquery to exclude those who have reserved such boat

```
SELECT DISTINCT s.sid
FROM Sailors s
WHERE s.age > 20 AND s.sid NOT IN (
    SELECT r.sid
    FROM Reserves r
    JOIN Boats b ON r.bid = b.bid
    WHERE b.bname LIKE '%thunder%'
);
```

Q4.b Discuss the Equijoin & Natural Join with suitable example.

Equijoin and **Natural Join** are both types of joins used in SQL to combine rows from two or more tables based on a related column between them. They serve similar purposes but differ in their specific operations and results.

1. Equijoin

Definition: An Equijoin is a type of join where tables are joined based on the equality of specified columns. It involves a condition where the values in one

VTU360.IN

column from the first table are matched with values in a column from the second table using the equality operator (=).

Example: Consider two tables, Employees and Departments: **Employees Table**:

emp_id	emp_name	dept_id
1	Alice	101
2	Bob	102
3	Carol	101

Departments Table:

dept_id	dept_name
101	HR
102	IT
103	Finance

To find the names of employees along with their department names, you can perform an equijoin on the dept_id column:

sql
Copy code

```
SELECT e.emp_name, d.dept_name
FROM Employees e
JOIN Departments d ON
e.dept_id = d.dept_id;
```

Result:

emp_name	dept_name
Alice	HR
Bob	IT
Carol	HR

Explanation: In this equijoin, the dept_id column is used to match rows between the Employees and Departments tables. The equality condition e.dept_id = d.dept_id is applied to combine relevant rows.

2. Natural Join

Definition: A Natural Join automatically joins tables based on columns with the same names and compatible data types in both tables. It eliminates duplicate columns from the result. The join is performed on all columns with the same names in both tables.

Example: Using the same Employees and Departments tables: **Employees Table**:

VTU360.IN

emp_id	emp_name	dept_id
1	Alice	101
2	Bob	102
3	Carol	101

Departments Table:

dept_id	dept_name
101	HR
102	IT
103	Finance

To find the names of employees along with their department names using a natural join: sql
Copy code

```
SELECT emp_name, dept_name
FROM Employees
NATURAL JOIN Departments;
```

Result:

emp_name	dept_name
Alice	HR
Bob	IT
Carol	HR

Explanation: The NATURAL JOIN automatically joins the Employees and Departments tables on the dept_id column, as it is common to both tables. The result set includes only one dept_id column, with duplicate columns being eliminated.

Key Differences

- **Join Condition**:
- **Equijoin**: Requires an explicit condition for equality using ON clause. You specify which columns to match.
- **Natural Join**: Automatically uses columns with the same names for the join condition. No need to specify the join condition explicitly.
- **Duplicate Columns**:
- **Equijoin**: Both tables' columns included in the join condition appear in the result set unless specified otherwise.
- **Natural Join**: Removes duplicate columns that are used for the join, showing each column only once in the result.
- **Flexibility**:
- **Equijoin**: More flexible because you can choose which columns to join on and how to handle them.

VTU360.IN

- **Natural Join**: Less flexible as it automatically joins on all columns with the same names and does not provide explicit control over the join conditions.

Q4.c Explain the relational algebra operation for set theory with examples.

Relational algebra operations based on set theory are fundamental in querying and manipulating relational databases. These operations operate on relations (tables) and produce new relations as results. Here's an overview of key relational algebra operations that are derived from set theory, along with examples for each:

1. Union (∪)

Definition: The union operation combines the tuples of two relations, removing duplicates. It requires that both relations have the same schema (i.e., the same number of attributes with the same domain). **Notation**:

R∪S

Example:

Consider two relations, **Students1** and **Students2**: **Students1**:

student_id	name
1	Alice
2	Bob

Students2:

student_id	name
2	Bob
3	Carol

The union of `Students1` and `Students2` is:

```
SELECT * FROM Students1

UNION

SELECT * FROM Students2;
```

Result:

student_id	name
1	Alice
2	Bob
3	Carol

2. Intersection (∩)

VTUSO.IN

Definition: The intersection operation returns the tuples that are common to both relations. Like union, it requires that both relations have the same schema.

Notation: R∩S

Example:

Using the same relations as above:

Students1:

student_id	name
1	Alice
2	Bob

Students2:

student_id	name
2	Bob
3	Carol

The intersection of Students1 and Students2 is:

```
SELECT * FROM Students1
INTERSECT
SELECT * FROM Students2;
```

Result:

student_id	name
2	

3. Difference (−)

Definition: The difference operation returns tuples that are present in the first relation but not in the second. Both relations must have the same schema.

Notation:R−S **Example**:

Using the same relations as above: **Students1**:

student_id	name
1	Alice
2	Bob

Students2:

student_id	name
2	Bob

VTU360.IN

3	Carol

The difference of **Students1** minus **Students2** is:

```
SELECT * FROM Students1
EXCEPT
SELECT * FROM Students2;
```

Result:

student_id	name
1	Alice

4. Cartesian Product (×)

Definition: The Cartesian product operation combines each tuple of one relation with each tuple of another relation. This operation results in a relation that includes all possible combinations of tuples from the two relations. **Notation**: R×S

Example:

Consider two relations: **Students**:

student_id	name
1	Alice
2	Bob

Courses:

course_id	course_name
101	Math
102	Science

The Cartesian product of **Students** and **Courses** is:

```
SELECT * FROM Students
CROSS JOIN Courses;
```

Result:

student_id	name	course_id	course_name
1	Alice	101	Math
1	Alice	102	Science
2	Bob	101	Math
2	Bob	102	Science

VTU360.IN

5. Rename (ρ)

Definition: The rename operation changes the name of a relation or its attributes. This operation is useful for providing more meaningful names or for resolving naming conflicts in queries involving multiple relations. **Nota'on**: ρnew_name(R)\rho_{new_name}(R)ρnew_name(R) or ρnew_name(A1,A2,…,An)(R)\rho_{new_name(A1, A2, \ldots, An)}(R)ρnew_name(A1,A2,…,An)(R)

Example:

Consider a relation **Employees**: **Employees**:

emp_id	emp_name
1	Alice
2	Bob

To rename **Employees** to **Staff** and the attribute **emp_name** to **name**, you can use: SELECT emp_id AS id, emp_name AS name FROM Employees; **Result**:

id	name
1	Alice
2	Bob

VTU360.IN

Q5.a Explain the Cursor & its properties in embedded SQL with an example.

A cursor is a database object used to retrieve and process rows from a query result set one at a time.

Properties of Cursors

1. **Declare**: Define the cursor with a `DECLARE` statement, specifying the SQL query.
2. **Open**: Use `OPEN` to execute the query and create the result set.
3. **Fetch**: Use `FETCH` to retrieve individual rows from the result set.
4. **Close**: Use `CLOSE` to release resources associated with the cursor.

5. **Deallocate**: Optionally use `DEALLOCATE` to remove the cursor definition and free resources.

Example

Here's a basic example using a cursor in embedded SQL:

```
-- Declare the cursor
DECLARE emp_cursor CURSOR FOR
SELECT emp_id, emp_name, salary FROM Employees;

-- Open the cursor
OPEN emp_cursor;

-- Fetch rows
FETCH NEXT FROM emp_cursor INTO @emp_id, @emp_name, @salary;

-- Process rows
WHILE @@FETCH_STATUS = 0
BEGIN
    -- Do something with the data (e.g., print it)
    PRINT 'ID: ' + CAST(@emp_id AS VARCHAR) + ', Name: ' + @emp_name;

    -- Fetch the next row
    FETCH NEXT FROM emp_cursor INTO @emp_id, @emp_name, @salary;
END;
```

VTU360.IN

-- Close the cursor

CLOSE emp_cursor;

-- Deallocate the cursor

DEALLOCATE emp_cursor;

Q5.b What is a Normalization? Explain the 1NF, 2NF & 3NF with examples.

Normalization is the process of organizing a database to reduce redundancy and improve data integrity. The goal is to ensure that the database structure is efficient and logical, and to eliminate anomalies in data handling. Normalization involves decomposing tables into smaller tables and defining relationships among them based on rules called **normal forms**.
Here's an explanation of the first three normal forms (1NF, 2NF, and 3NF) with examples:

1. First Normal Form (1NF)

Definition: A table is in **First Normal Form (1NF)** if all the columns contain atomic (indivisible) values, and each column contains only one type of data. Additionally, each column must have a unique name, and the order in which data is stored does not matter.
Example:
Consider a table storing information about students and their enrolled courses:
Student_Courses:

student_id	student_name	courses
1	Alice	Math, Science
2	Bob	Science, History

This table is not in 1NF because the **courses** column contains multiple values.
To convert this to 1NF:
Create a separate row for each course. **Student_Courses_1NF**:

student_id	student_name	course
1	Alice	Math
1	Alice	Science
2	Bob	Science
2	Bob	History

VTUSOLN

2. Second Normal Form (2NF)

Definition: A table is in **Second Normal Form (2NF)** if it is in 1NF and all nonkey attributes are fully functionally dependent on the entire primary key. In other words, there should be no partial dependency of any column on a subset of a composite primary key.

Example:

Consider a table with the following schema and data: **Orders**:

order_id	product_id	product_name	quantity
1	101	Laptop	2
1	102	Mouse	1
2	101	Laptop	1

Here, **order_id** and **product_id** together form the composite primary key. **product_name** is dependent only on **product_id** and not on **order_id**.

To convert this to 2NF:

Separate the product information into its own table. **Orders**:

order_id	product_id	quantity
1	101	2
1	102	1
2	101	1

Products:

product_id	product_name
101	Laptop
102	Mouse

3. Third Normal Form (3NF)

Definition: A table is in **Third Normal Form (3NF)** if it is in 2NF and all the attributes are functionally dependent only on the primary key, and not on any other non-key attributes. This means there should be no transitive dependency.

Example:

Consider a table with the following schema: **Employees**:

emp_id	emp_name	department	department_manager
1	Alice	HR	John Doe

VTU360.IN

2	Bob	IT	Jane Smith

In this table, **department_manager** depends on **department**, which is a non-key attribute. **To convert this to 3NF**:
Remove the transitive dependency by creating a separate table for department details.

Employees:

emp_id	emp_name	department
1	Alice	HR
2	Bob	IT

Departments:

department	department_manager
HR	John Doe
IT	Jane Smith

Q5.C Explain informal design guidelines for relational schema design.

---

1. Minimize Redundancy

Guideline: Avoid duplicating data across tables. Redundant data increases storage requirements and can lead to inconsistencies. Ensure that each piece of information is stored only once.

- **Example**: Instead of storing a customer's address in every order record, store the address in a separate `Customers` table and reference it in the `Orders` table.

2. Avoid Insertion, Update, and Deletion Anomalies

Guideline: Design the schema to prevent anomalies that can occur when inserting, updating, or deleting records. This typically involves ensuring that the schema is normalized to a suitable level (usually 3NF).

- **Example**: If you have a table where updating a customer's address could require multiple updates due to redundant storage, it's better to have a separate `Addresses` table and link it with foreign keys.

3. Ensure Data Integrity

Guideline: Use constraints to ensure that the data in the database is accurate and reliable. This includes primary keys to uniquely identify records, foreign keys to maintain referential integrity, and other constraints to enforce valid data.

VTU360.IN

- **Example**: Use foreign keys to ensure that every `order` in the `Orders` table refers to a valid `customer` in the `Customers` table.

4. Design for Flexibility and Simplicity

Guideline: The schema should be designed to be simple and flexible. Avoid overly complex designs and ensure the schema can accommodate future changes with minimal modifications.

- **Example**: Instead of combining multiple types of information into one table, use separate tables with clear relationships. For example, use separate tables for `Employees` and `Departments` with a clear foreign key relationship.

VTU360.IN